Spark MapReduce Analysis

Big Data Analytics Final Project

Team: Emre Akyol, Harmanpreet Chauhan, Mohamed Nasr

This notebook demonstrates distributed data processing using Apache Spark MapReduce operations.

In [1]:
```
import json
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

SPARK_AVAILABLE = False
try:
    from pyspark import SparkContext, SparkConf
    SPARK_AVAILABLE = True
    print("PySpark available")
except ImportError:
    print("Using Python MapReduce fallback")
```

```
PySpark available
```

In [2]:
```
if SPARK_AVAILABLE:
    conf = SparkConf().setAppName('CryptoAnalysis').setMaster('local[*]')
    conf.set('spark.ui.showConsoleProgress', 'false')
    sc = SparkContext.getOrCreate(conf=conf)
    sc.setLogLevel('ERROR')
    print(f"Spark {sc.version} initialized")
```

```
Spark 3.5.0 initialized
```

In [3]:
```
with open('./resources/data/crypto-prices.json', 'r') as f:
    data = json.load(f)
cryptos = data['cryptocurrencies']
print(f"Loaded {len(cryptos)} cryptocurrencies")
```

```
Loaded 6 cryptocurrencies
```

MapReduce 1: Average Sentiment

In [4]:
```
if SPARK_AVAILABLE:
    rdd = sc.parallelize(cryptos)
    avg_sentiment = rdd.map(lambda x: (x['symbol'], x.get('socialSentiment', 0))).c
else:
    avg_sentiment = {c['symbol']: c.get('socialSentiment', 0) for c in cryptos}

print("Sentiment per Crypto:")
for sym, sent in avg_sentiment.items():
```

```
    print(f"  {sym}: {sent:.3f}")
```

```
Sentiment per Crypto:
  BTC: 0.020
  ETH: 0.030
  XRP: 0.080
  SOL: 0.040
  DOGE: 0.120
  ADA: 0.060
```

MapReduce 2: Market Metrics

In [5]:
```
if SPARK_AVAILABLE:
    rdd = sc.parallelize(cryptos)
    total_market_cap = rdd.map(lambda x: float(x['marketCap'])).reduce(lambda a,b:
    total_volume = rdd.map(lambda x: float(x['volume24h'])).reduce(lambda a,b: a+b)
else:
    total_market_cap = sum(float(c['marketCap']) for c in cryptos)
    total_volume = sum(float(c['volume24h']) for c in cryptos)

btc = next((c for c in cryptos if c['symbol'] == 'BTC'), None)
btc_dominance = (float(btc['marketCap']) / total_market_cap * 100) if btc else 0

print(f"Total Market Cap: ${total_market_cap/1e12:.2f}T")
print(f"Total Volume: ${total_volume/1e9:.2f}B")
print(f"BTC Dominance: {btc_dominance:.1f}%")
```

```
Total Market Cap: $2.44T
Total Volume: $54.86B
BTC Dominance: 74.6%
```

MapReduce 3: Volatility Classification

In [6]:
```
def classify_vol(v): return 'Low' if v < 0.02 else 'Medium' if v < 0.05 else 'High'

if SPARK_AVAILABLE:
    rdd = sc.parallelize(cryptos)
    vol_class = rdd.map(lambda x: (classify_vol(x['volatility']), x['symbol'])).gro
else:
    vol_class = {}
    for c in cryptos:
        cat = classify_vol(c['volatility'])
        vol_class.setdefault(cat, []).append(c['symbol'])

print("Volatility Classification:")
for cat in ['Low', 'Medium', 'High']:
    if cat in vol_class:
        print(f"  {cat}: {', '.join(vol_class[cat])}")
```

```
Volatility Classification:
  Low: BTC, ETH, SOL
  Medium: XRP, ADA
  High: DOGE
```

MapReduce 4: Price Performance

In [7]:
```
def classify_perf(c): return 'Strong Gain' if c > 5 else 'Gain' if c > 0 else 'Loss

if SPARK_AVAILABLE:
    rdd = sc.parallelize(cryptos)
    perf_class = rdd.map(lambda x: (classify_perf(x['change24h']), x['symbol'])).gr
else:
    perf_class = {}
    for c in cryptos:
        cat = classify_perf(c['change24h'])
        perf_class.setdefault(cat, []).append(c['symbol'])

print("Price Performance:")
for cat in ['Strong Gain', 'Gain', 'Loss', 'Strong Loss']:
    if cat in perf_class:
        print(f"  {cat}: {', '.join(perf_class[cat])}")
```

```
Price Performance:
  Strong Gain: DOGE
  Gain: BTC, ETH, XRP, SOL, ADA
```

MapReduce 5: Sentiment-Volume Score

In [8]:
```
if SPARK_AVAILABLE:
    rdd = sc.parallelize(cryptos)
    scores = rdd.map(lambda x: (x['symbol'], x.get('socialSentiment',0) * x['volume
else:
    scores = [(c['symbol'], c.get('socialSentiment',0) * c['volume24h']/1e9) for c

sorted_scores = sorted(scores, key=lambda x: x[1], reverse=True)
print("Engagement Scores:")
for sym, score in sorted_scores:
    print(f"  {sym}: {score:.2f}")
```

```
Engagement Scores:
  BTC: 0.63
  ETH: 0.41
  DOGE: 0.27
  XRP: 0.25
  SOL: 0.13
  ADA: 0.04
```

Save Results

In [9]:
```
results = {
    'timestamp': datetime.now().isoformat(),
    'engine': 'Spark' if SPARK_AVAILABLE else 'Python',
    'results': {
        'average_sentiment': avg_sentiment,
        'market_metrics': {'total_market_cap': float(total_market_cap), 'total_volu
        'volatility_classification': {k: list(v) for k,v in vol_class.items()},
```

```
        'price_performance': {k: list(v) for k,v in perf_class.items()},
        'engagement_scores': [{'symbol': s, 'score': float(sc)} for s,sc in sorted_
    },
    'metadata': {'cryptos': len(cryptos), 'operations': 5}
}

with open('./resources/data/spark-analysis-results.json', 'w') as f:
    json.dump(results, f, indent=2)
print("Saved to spark-analysis-results.json")
```

```
Saved to spark-analysis-results.json
```

In [10]:
```
if SPARK_AVAILABLE:
    sc.stop()
    print("Spark stopped")
```

```
Spark stopped
```

Summary

5 MapReduce operations performed: sentiment aggregation, market metrics, volatility classification, price performance, and engagement scoring.